

Claude Code Enterprise Permissions Matrix

A quick-reference guide for configuring permissions at every level of the Claude Code settings hierarchy. Use this matrix to ensure the right permissions are enforced by the right people at the right scope.

Settings Hierarchy (Priority Order)

Claude Code evaluates permissions in a strict priority order. Higher-priority settings always win. This means an enterprise admin can enforce deny rules that no individual developer can override.

Priority	Settings Level	File Location	Who Controls It	Committed to Git?	Can Developer Override?	Use Case
1 (Highest)	Enterprise Managed	managed-settings.json (admin-deployed)	IT Admin / Security	No	No — enforced	Org-wide security policies, mandatory deny rules, compliance requirements
2	Command-Line Flags	CLI arguments (per-session)	Individual Developer	N/A (ephemeral)	Yes (temporary)	One-off overrides for specific sessions, testing, demos
3	Local Project	.claude/settings.local.json	Individual Developer	No (gitignored)	Yes	Personal preferences per project, local paths, dev-specific allow rules
4	Shared Project	.claude/settings.json	Tech Lead / Team	Yes — team standard	Yes (but shouldn't)	Team-wide permissions, shared deny rules, project-specific tool access
5 (Lowest)	Global User	~/.claude/settings.json	Individual Developer	No (personal)	Yes	Personal defaults across all projects, preferred model, editor prefs

Key principle: Enterprise managed settings (Priority 1) cannot be overridden by any lower level. This is how you prevent a developer from going rogue — even if they modify .claude/settings.json or their global settings, the managed policy still enforces your deny rules.

Recommended Permissions by Role

Map your team roles to permission configurations. The table below shows recommended settings for each tool type across common enterprise roles. Adapt to your organization's risk tolerance.

Tool / Action	Junior Dev	Senior Dev	Tech Lead	DevOps / SRE	CI/CD Pipeline	Where to Configure
Bash (general)	ASK	ASK	ASK	ASK	ALLOW (bypass)	Shared project settings
Bash(rm:*)	DENY	DENY	ASK	ASK	DENY	Managed settings
Bash(cat .env:*)	DENY	DENY	DENY	DENY	DENY	Managed settings
Bash(curl:*) / Bash(wget:*)	DENY	DENY	ASK	ASK	DENY	Managed settings
Bash(git push:*)	DENY	ASK	ALLOW	ALLOW	ALLOW	Shared project settings
Bash(git commit:*)	ASK	ALLOW	ALLOW	ALLOW	ALLOW	Shared project settings
Bash(npm install:*)	ASK	ASK	ALLOW	ALLOW	ALLOW	Shared project settings
Bash(docker:*)	DENY	ASK	ASK	ALLOW	ALLOW	Shared project settings
Read(/src/**)	ALLOW	ALLOW	ALLOW	ALLOW	ALLOW	Shared project settings
Read(~/.ssh/**)	DENY	DENY	DENY	DENY	DENY	Managed settings
Read(~/.aws/**)	DENY	DENY	DENY	ASK	DENY	Managed settings
Write(/src/**)	ASK	ALLOW	ALLOW	ALLOW	ALLOW	Shared project settings
Write(.env*)	DENY	DENY	DENY	DENY	DENY	Managed settings
Edit(/src/**)	ASK	ALLOW	ALLOW	ALLOW	ALLOW	Shared project settings
WebFetch(*)	DENY	ASK	ASK	ASK	DENY	Managed settings
mcp_ _ * (all MCP)	DENY	ASK	ASK	ASK	ASK	Managed settings (allowlist)
Skill(*)	ASK	ALLOW	ALLOW	ALLOW	ALLOW	Shared project settings

ALLOW = Permitted without prompt

ASK = Requires developer approval each time

DENY = Blocked entirely, cannot override

Managed = Enforced at enterprise/admin level

8 Deny Rules Every Enterprise Should Have

These rules should be configured in managed-settings.json (enterprise level) so they cannot be overridden by individual developers. They protect against the most common security risks in Claude Code deployments.

#	Deny Rule	What It Blocks	Why It Matters	Risk Without It
1	Bash(cat .env:*)	Reading .env files via shell	Prevents Claude from accessing API keys, database passwords, and secrets stored in environment files	Full credential exposure — every secret in .env readable by Claude and potentially logged
2	Read(~/.ssh/**)	Reading SSH private keys	SSH keys grant access to servers, Git repos, and infrastructure. Never let an AI agent read them	Remote server compromise if keys are exposed or logged
3	Read(~/.aws/**)	Reading AWS credentials	AWS credentials can provision infrastructure, access S3 data, and incur costs. Same applies to GCP/Azure credential paths	Cloud account takeover, data exfiltration, surprise billing
4	Bash(curl:*)	Arbitrary HTTP requests from shell	Prevents data exfiltration via outbound HTTP calls. Claude could be prompt-injected into sending data to external servers	Data exfiltration, command-and-control communication
5	Bash(wget:*)	Downloading files from the internet	Prevents downloading potentially malicious scripts or packages outside of your approved package manager	Supply chain attacks, malicious script execution
6	Bash(rm -rf:*)	Recursive force-delete	An AI agent with recursive delete capability could destroy entire directory trees in one command	Catastrophic data loss, project destruction
7	Write(.env*)	Writing/modifying environment files	Prevents Claude from adding, changing, or corrupting secrets. Also prevents creating new .env files that bypass gitignore	Credential tampering, backdoor secrets injection
8	Bash(chmod 777:*)	Setting world-readable/writable permissions	Prevents Claude from making files or directories accessible to all users on the system	Privilege escalation, unauthorized file access

Example: managed-settings.json (Enterprise Admin)

```
{
  "permissions": {
    "deny": [
      "Bash(cat .env:*)",
      "Bash(curl:*)",
      "Bash(wget:*)",
      "Bash(rm -rf:*)",
      "Bash(chmod 777:*)",
      "Read(~/.ssh/**)",
      "Read(~/.aws/**)",
      "Write(.env*)"
    ]
  }
}
```

```
}

```

Deployment: Place managed-settings.json in the enterprise-managed configuration path for your deployment. This file is controlled by IT/Security admins and cannot be modified by developers. All eight rules above apply to every engineer, every session, every project — no exceptions.

Permission Modes: When to Use Each

Mode	Behavior	When to Use	When NOT to Use	Risk Level
Normal	Claude asks permission for each tool use not explicitly allowed or denied	Daily development work. Default for all team members. Recommended for first 2–4 weeks of adoption	CI/CD pipelines (too many prompts)	LOW
Plan	Claude can only read and analyze. No writes, no execution. Generates a plan for review	Architecture design, code review, codebase exploration, onboarding new team members	When you need Claude to execute changes	LOWEST
Auto-Accept	Claude executes allowed actions without prompts. Denied actions still blocked	Trusted workflows after plan review. Senior devs on well-configured repos with strong deny rules in place	New repos, junior devs, repos without deny rules, sensitive codebases	MEDIUM
Bypass	All permissions bypassed. Full autonomous execution with no prompts	ONLY in isolated CI/CD containers (Docker). Fully sandboxed environments with no access to real credentials	NEVER on developer machines. NEVER on repos with real credentials. NEVER without container isolation	HIGHEST

Quick Decision Framework

Use this flowchart to choose the right permission mode for any scenario:

Scenario	Recommended Mode	Required Safeguards
New team member exploring the codebase for the first time	Plan	CLAUDE.md in place, shared settings committed
Daily development — writing features, fixing bugs	Normal	Deny rules configured, CLAUDE.md current, sandbox enabled
Senior dev executing a reviewed plan on a well-governed repo	Auto-Accept	All 8 deny rules active, managed settings deployed, hooks running tests
Automated PR review in GitHub Actions	Auto-Accept (in container)	Docker isolation, no real credentials mounted, read-only source
Full autonomous CI build and test pipeline	Bypass (Docker only)	Fully isolated container, no network access to prod, ephemeral environment, no secrets
Production deployment scripts	NEVER use Claude Code	Production deployments should use your existing CI/CD pipeline, not an AI agent

About This Resource

This permissions matrix is a bonus resource from **Claude Code Enterprise Mastery: Ship Faster, Ship Safer** by Ritesh Vajariya, Founder of AI Guru®.

For the full course with live demos of every configuration shown here, visit:

Udemy: Search "Claude Code Enterprise Mastery"

Platform: aiguru.academy

Enterprise workshops: Contact via aiguru.one

Version 1.0 — March 2026 | © 2026 AI Guru®. All rights reserved.